

# Lifecycle and Performance Analysis of Memory Allocation Strategies Using Valgrind and perf

Md Maroof Hasan Khan  
MT2025070

Piyush Singh  
MT2025088

## I. INTRODUCTION

Understanding how memory behaves during program execution is important for analyzing both performance and efficiency in modern systems. Most applications rely heavily on dynamic memory allocation through functions such as `malloc()`, `calloc()`, `realloc()` and `free()`, but the behavior of these allocations is often hidden beneath the abstraction provided by programming languages and standard libraries.

Traditional memory analysis usually focuses on final results such as total memory usage or memory leaks. While these metrics are useful, they do not explain how memory evolves during execution or how allocation strategies influence runtime behavior. In practice, many performance problems arise not because a program uses too much memory, but because of how memory is allocated, accessed, reused and released throughout execution.

Modern performance issues are strongly connected to allocator behavior, fragmentation, cache locality, page allocation overhead and memory-access efficiency. Different data structures generate fundamentally different memory layouts, and these layouts directly affect how efficiently the CPU, cache hierarchy and operating system interact with the program.

In this project, we study the lifecycle of memory allocation rather than only the final state of memory. We observe:

- How memory grows during execution
- How different allocation strategies behave
- How allocator reuse changes latency over time
- How fragmentation affects performance
- How memory layout influences hardware-level execution

Dynamic memory allocation is deeply connected to operating-system level mechanisms such as heap expansion, virtual memory management, page allocation and allocator reuse. Therefore, studying memory allocation provides insight not only into software-level efficiency, but also into how applications interact with the underlying operating system and hardware.

To perform this analysis, two major tools were used:

- **Valgrind** for software-level memory profiling
- **perf** for hardware-level performance analysis

Valgrind Massif helps visualize heap growth over time, while Memcheck validates correctness by detecting memory leaks and invalid accesses. `perf` complements Valgrind by measuring hardware performance counters such as instructions executed, cache references, cache misses and execution time.

To study different allocation behaviors, two categories of programs were implemented:

- Graph-based structures using adjacency list and adjacency matrix representations
- Queue-based programs (`queue_gpt`, `queue_claude` and `queue_grok`) simulating dynamic allocation lifecycles

The adjacency-list implementation represents fragmented pointer-heavy allocation behavior, while the adjacency matrix represents contiguous allocation behavior. The queue programs simulate dynamic workloads involving repeated allocations and deallocations.

The main goal of this project is to understand the trade-offs between different allocation strategies in terms of:

- Heap growth
- Allocation latency
- Fragmentation
- Cache locality
- Scalability
- Variability
- Hardware-level execution behavior

Instead of focusing on a single metric, the project attempts to explain why these differences occur and how memory allocation fundamentally shapes runtime behavior.

## II. PROBLEM STATEMENT

Most traditional memory analysis techniques focus primarily on final results such as total heap usage or memory leak detection. While these metrics are important, they do not provide a complete understanding of how memory behaves dynamically during execution.

Two programs may consume similar amounts of memory and produce identical outputs while exhibiting completely different runtime behaviors internally. The differences often arise from allocation strategy, memory layout, allocator reuse behavior and hardware-level memory-access patterns.

The primary objective of this project is to study how different allocation strategies influence:

- Memory growth patterns during execution
- Allocation latency and variability
- Fragmentation and allocator overhead
- Cache locality and hardware efficiency
- Scalability under increasing workloads
- Memory reuse behavior

Different data structures naturally produce different allocation behaviors.

Pointer-based structures such as adjacency lists depend on a large number of small allocations. These allocations are typically distributed across the heap and may lead to:

- Fragmentation
- Pointer chasing
- Poor spatial locality
- Increased allocator metadata overhead

In contrast, contiguous structures such as adjacency matrices use a very small number of large allocations. These improve locality and predictability but may suffer from extremely large memory requirements and poor scalability.

Dynamic workloads such as queues introduce additional complexity because allocations and deallocations occur continuously during execution. Modern allocators internally reuse previously freed memory using freelists and thread-local caches, meaning allocator behavior may evolve significantly as workload size increases.

Another major challenge addressed in this project is variability in measurements. Allocation latency is not deterministic. Even identical workloads executed multiple times may produce different results because of:

- Operating-system scheduling
- Allocator state
- Page faults
- CPU cache state
- TLB state
- Background system activity
- Hardware frequency scaling

Understanding this variability is important because systems measurements are probabilistic rather than perfectly deterministic.

To study scalability, multiple workloads were tested:

- Graph workloads using increasing numbers of vertices and edges
- Queue workloads using 100K, 1M and 10M allocations

Overall, the project aims not only to compare memory usage between programs, but also to explain why these differences occur by relating allocator behavior to fragmentation, locality, scalability and hardware-level execution effects.

### III. RELATED WORK

This work is based primarily on the Valgrind framework introduced by Nethercote and Seward in “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation”.

Before Valgrind was introduced, memory debugging tools were limited and often inaccurate. Detecting subtle bugs such as invalid memory accesses, use-after-free errors and uninitialized reads was extremely difficult.

Valgrind introduced the concept of dynamic binary instrumentation.

Valgrind does not execute the program directly on physical hardware. Instead, machine instructions are intercepted and translated into an intermediate representation (IR). Additional

instrumentation instructions are inserted into the IR before execution on a synthetic CPU provided by the Valgrind core engine.

Internally, Valgrind performs the following operations:

- 1) Intercepts machine code
- 2) Converts instructions into IR
- 3) Adds instrumentation instructions
- 4) Executes instrumented code on a virtual CPU

This allows Valgrind to precisely monitor:

- Heap allocations
- Memory accesses
- Invalid reads and writes
- Heap growth over time
- Memory leaks

Massif and Memcheck are both built on top of the same Valgrind instrumentation framework.

Massif tracks:

- Heap growth
- Peak memory usage
- Allocation sources
- Heap evolution during runtime

Memcheck validates:

- Correct allocation/free matching
- Invalid memory accesses
- Use-after-free errors
- Memory leaks

A major limitation of Valgrind is its extremely high overhead. Since every instruction is translated and instrumented, execution becomes significantly slower. Additionally, Valgrind cannot accurately model hardware-level behavior such as:

- CPU cache hierarchy
- Branch prediction
- Hardware prefetching
- TLB effects
- Pipeline behavior

The paper “How to Shadow Every Byte of Memory Used by a Program” explains how Valgrind internally tracks memory using shadow memory.

For every real memory byte, Valgrind maintains metadata in a corresponding shadow-memory structure. The metadata tracks:

- Allocation status
- Accessibility permissions
- Initialization state

Shadow memory enables extremely precise error detection but introduces substantial memory and execution overhead.

To complement Valgrind, this project also uses Linux perf.

Unlike Valgrind, perf uses hardware performance counters directly available inside modern CPUs. These counters provide low-overhead measurements of:

- Instructions executed
- Cache references
- Cache misses
- CPU cycles

- Execution time

perf therefore allows analysis of hardware-level memory behavior which cannot be accurately modeled by Valgrind.

In this project, Valgrind and perf complement each other:

- Valgrind explains how memory is allocated and managed
- perf explains how efficiently memory is accessed by hardware

The combination provides a much deeper systems-level understanding of memory-allocation behavior.

#### IV. METHODOLOGY

The goal of this project is to analyze memory allocation behavior from both software-level and hardware-level perspectives. To achieve this, controlled programs were implemented in C and analyzed using Valgrind and perf.

##### A. Valgrind Analysis

Valgrind was used to analyze software-level memory behavior.

Two Valgrind tools were used:

1) *Massif*: Massif measures:

- Heap growth over time
- Peak heap memory usage
- Allocation sources
- Memory lifecycle patterns

Massif periodically records snapshots of live heap memory during execution. The generated graphs represent currently allocated heap memory rather than allocation frequency.

This distinction is important because memory may be freed internally while the allocator still retains heap pages.

2) *Memcheck*: Memcheck was used to validate program correctness.

It detects:

- Memory leaks
- Invalid reads and writes
- Double frees
- Use-after-free errors

This ensures that experimental measurements are not affected by memory-management bugs.

##### B. perf Analysis

perf was used for hardware-level analysis.

The following metrics were collected using `perf stat`:

- Instructions executed
- Cache references
- Cache misses
- Execution time

These metrics help explain how efficiently memory is accessed and how allocation strategies interact with the CPU cache hierarchy.

##### C. Allocator and Operating-System Interaction

Although `malloc()` appears as a simple library function, modern memory allocation involves multiple layers.

User-space allocators such as glibc `malloc` manage:

- Heap chunks
- Allocation metadata
- Freelist reuse
- Thread-local caches
- Arenas

Small allocations are often served directly from allocator freelists without requiring kernel interaction.

Large allocations may internally trigger:

- `mmap()`
- Heap expansion using `brk()`

Modern operating systems use virtual memory and demand paging. Therefore, `malloc()` does not necessarily allocate physical RAM immediately. Physical pages are commonly assigned lazily when memory is first accessed, causing page faults during execution.

The Linux kernel internally uses allocators such as the buddy allocator and SLUB allocator for page management and kernel-object allocation. Although user programs do not directly invoke these allocators, allocation behavior indirectly influences kernel-level memory management.

1) *glibc Allocator Internals*: Modern glibc `malloc` implementations internally use multiple allocation optimizations to reduce allocation overhead.

Small allocations are commonly handled through thread-local caches known as `tcaches`. Freed memory chunks are stored in size-specific freelists local to each thread. When a subsequent `malloc` request of the same size occurs, the allocator can often satisfy the request directly from the `tcache` without requiring synchronization or kernel interaction.

Very small allocations are additionally optimized using `fastbins`. `Fastbins` allow recently freed chunks to be rapidly recycled without immediate coalescing, significantly improving allocation throughput for workloads involving repeated small allocations.

The adjacency-list and queue workloads are strongly influenced by these allocator optimizations because they repeatedly allocate similarly sized nodes. As allocator freelists become populated during execution, later allocations are increasingly satisfied from previously freed chunks, reducing allocation latency.

glibc `malloc` also internally switches to `mmap()` for sufficiently large allocations. Large adjacency-matrix allocations therefore likely bypassed the traditional heap allocator entirely and were managed as independent virtual-memory mappings. This explains why adjacency-matrix allocations exhibited high individual allocation latency but relatively low variability due to the small number of allocations performed.

##### D. Program Design

The programs were intentionally designed to isolate allocation behavior.

Complex real-world software often mixes many unrelated behaviors, making allocator analysis difficult. Controlled workloads help expose the impact of allocation strategy more clearly.

1) *Adjacency List*: The adjacency-list implementation uses:

- Pointer-linked nodes
- One malloc call per edge insertion
- Large numbers of small allocations

This creates fragmented pointer-heavy heap behavior.

2) *Adjacency Matrix*: The adjacency matrix uses:

- Large contiguous memory allocation
- Flattened matrix representation
- Very few malloc calls

This isolates contiguous allocation behavior and spatial locality.

3) *Queue Programs*: Three queue implementations were analyzed:

- queue\_gpt
- queue\_claude
- queue\_grok

The queue workloads continuously:

- Allocate memory
- Free memory
- Reuse memory
- Grow and partially shrink

This simulates dynamic memory lifecycles more realistically than graph structures.

The queue programs were designed to observe:

- Allocator reuse
- Freelist behavior
- Heap oscillation
- Allocation stabilization
- Instrumentation effects

## E. Workloads

Different workloads were used to study scalability.

1) *Graph Workloads*: **Small workload**

- Vertices: 500
- Edges: 50000

**Medium workload**

- Vertices: 5000
- Edges: 500000

**Large workload**

- Vertices: 50000
- Edges: 5000000

2) *Queue Workloads*:

- 100K allocations
- 1M allocations
- 10M allocations

As workload size increases, allocator freelists become populated and memory reuse improves. Larger workloads therefore expose allocator stabilization behavior more clearly.

## F. Latency Measurement

Each malloc call was individually timed using:

```
clock_gettime(CLOCK_MONOTONIC)
```

Latency was measured in nanoseconds.

For each program:

- Total latency
- Average latency
- Maximum latency
- Minimum latency

were recorded.

This allows analysis of both average and worst-case allocation behavior.

## G. Variability Consideration

Each experiment was executed multiple times to study variability.

This is important because systems measurements are inherently probabilistic.

Variability may arise from:

- Allocator state
- OS scheduling
- Page faults
- Cache state
- Background activity
- TLB effects

The experiments also study how increasing sample size improves statistical stability.

## V. RESULTS AND ANALYSIS

### A. Adjacency List

The adjacency-list implementation uses pointer-based graph representation.

For each edge insertion:

```
malloc(sizeof(Node))
```

is executed.

With:

- V = 5000
- E = 500000

more than 500000 individual allocations are generated.

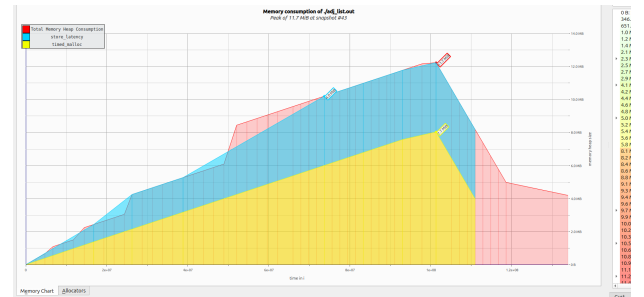


Fig. 1: Massif output for adjacency list

The Massif graph shows gradual heap growth because allocations occur incrementally during graph construction.

The fragmented nature of the adjacency list creates non-contiguous heap layout. Neighboring graph nodes are often located far apart in memory.

This produces:

- Fragmentation
- Allocator metadata overhead
- Pointer chasing
- Poor spatial locality

During traversal, the CPU repeatedly dereferences pointers:

node -> next -> next

which negatively affects cache efficiency.

Modern CPUs transfer memory in cache lines, typically 64 bytes at a time. During adjacency-list traversal, dereferencing a node pointer may require loading an entire cache line even though only a small portion of the line contains useful data.

Because neighboring nodes are often scattered throughout the heap, subsequent pointer traversals frequently trigger additional cache misses. This pointer-chasing behavior reduces spatial locality and limits the effectiveness of hardware prefetchers.

We hypothesized that the adjacency list would achieve lower memory usage but exhibit poorer cache behavior due to fragmented allocation patterns. The perf results confirmed this expectation through higher cache-miss rates compared to the adjacency matrix.

Massif results:

- Peak memory usage: ~11.7 MB
- Total malloc calls: 500002

Latency results:

| Run | Avg Latency (ns) | Max Latency (ns) |
|-----|------------------|------------------|
| 1   | 41.20            | 211558           |
| 2   | 68.71            | 49173            |
| 3   | 34.56            | 117141           |
| 4   | 45.80            | 102561           |
| 5   | 76.95            | 224830           |

Adjacency list latency variability

Average latency remains relatively stable, but occasional spikes become extremely large.

These spikes occur when the allocator must:

- Expand heap arenas
- Request new pages from the kernel
- Handle page faults
- Synchronize allocator metadata

The adjacency list is memory efficient and scalable, but suffers from fragmentation and poor locality.

### B. Adjacency Matrix

The adjacency matrix uses contiguous memory allocation.

Internally, the matrix is implemented using a flattened array:

```
matrix[row * V + col]
```

This creates a single large contiguous memory block.

We hypothesized that contiguous allocation would improve cache locality and reduce cache-miss rates because neighboring matrix elements reside close together in memory.

Sequential traversal allows multiple matrix entries to fit within the same cache line, significantly improving cache utilization. Hardware prefetchers can also predict future accesses more effectively during linear traversal patterns.

The perf results confirmed this expectation, with the adjacency matrix achieving the lowest cache-miss percentage among all workloads.

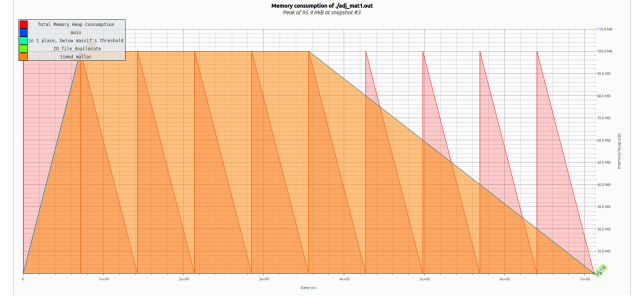


Fig. 2: Massif output for adjacency matrix

The Massif graph shows sudden heap growth followed by a stable plateau.

Unlike adjacency lists, the matrix performs very few allocations.

Massif results:

- Peak memory: ~95.4 MB
- Total malloc calls: 10

Large allocations may internally trigger mmap() rather than traditional heap expansion.

Latency results:

| Run | Avg Latency (ns) | Max Latency (ns) |
|-----|------------------|------------------|
| 1   | 22767            | 38031            |
| 2   | 19730            | 26299            |
| 3   | 21478            | 25577            |
| 4   | 22525            | 41558            |
| 5   | 24516            | 38201            |

Adjacency matrix latency variability

Although individual allocations are expensive, variability is smaller because the number of allocations is very low.

Contiguous memory improves:

- Spatial locality
- Cache efficiency
- Hardware prefetching
- TLB locality

However, computational cost grows quadratically:

$$O(V^2)$$

Even unused matrix entries are traversed.

This causes extremely large instruction counts despite excellent cache locality.

At  $V = 50000$ , memory requirements exceeded approximately 10 GB and resulted in segmentation failure.

This demonstrates poor scalability for large sparse graphs.

#### C. Comparison: Adjacency List vs Matrix

| Metric       | Adjacency List | Adjacency Matrix |
|--------------|----------------|------------------|
| Malloc Calls | 500002         | 10               |
| Peak Memory  | 7.7 MB         | 34.4 MB          |
| Avg Latency  | 40–80 ns       | ~20000 ns        |
| Max Latency  | 200000+ ns     | ~40000 ns        |
| Overhead     | High           | Low              |
| Locality     | Poor           | Good             |
| Scalability  | Good           | Poor             |

Adjacency list vs adjacency matrix

The adjacency list is memory efficient and scalable but suffers from fragmentation and poor locality.

The adjacency matrix provides excellent locality and stable allocation behavior but becomes infeasible for very large graphs because memory grows quadratically.

#### D. Queue Programs Analysis

The queue programs simulate dynamic memory lifecycles. Unlike graph structures, the queues continuously:

- Allocate memory
- Free memory
- Reuse previously freed memory
- Grow and partially shrink

This better represents real systems workloads.

1) *queue\_gpt*: *queue\_gpt* exhibits smooth and stable heap growth.

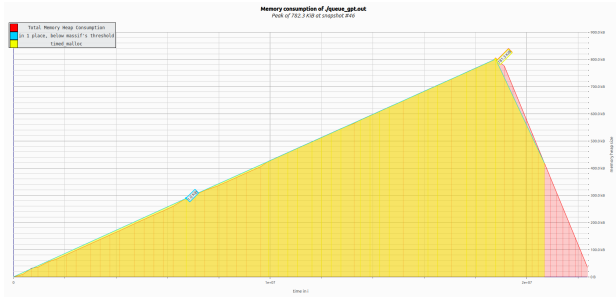


Fig. 3: Massif output for *queue\_gpt*

Latency results:

| Workload | Avg Latency (ns) |
|----------|------------------|
| 100K     | 46               |
| 1M       | 51               |
| 10M      | ~30              |

*queue\_gpt* latency

We hypothesized that larger queue workloads would reduce average allocation latency because allocator freelists and tcaches become populated during execution. The experimental results confirmed this expectation, with average allocation latency decreasing significantly as workload size increased.

This reduces average latency and stabilizes execution.

The heap graph appears smoother because allocator reuse internally masks many deallocation effects.

2) *queue\_claude*: *queue\_claude* showed more stable but heavier execution.

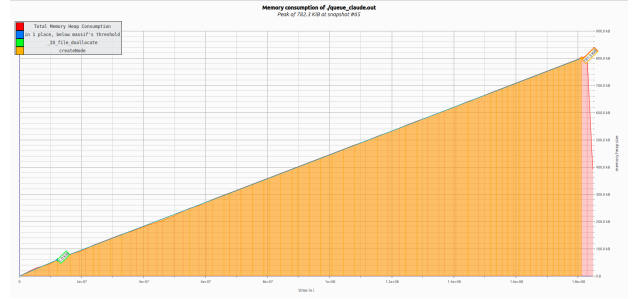


Fig. 4: Massif output for *queue\_claude*

Latency results:

| Workload | Avg Latency (ns) |
|----------|------------------|
| 100K     | 83–87            |
| 1M       | 42               |
| 10M      | 41               |

*queue\_claude* latency

*queue\_claude* introduced additional instrumentation and logging overhead.

Specifically, `printf()` calls inside allocation paths significantly increased execution cost.

This demonstrates the observer effect: measurement infrastructure itself can alter the runtime behavior being analyzed.

This phenomenon resembles the classic Heisenbug problem in systems debugging, where the act of observing a system changes its behavior.

The perf results showed approximately 740 million executed instructions for *queue\_claude* compared to roughly 56 million instructions for *queue\_gpt*. This indicates that debugging instrumentation introduced more than an order-of-magnitude increase in computational work.

These results demonstrate that excessive logging inside critical allocation paths can significantly distort performance measurements and produce misleading profiling data.

3) *queue\_grok*: *queue\_grok* most clearly visualized allocator lifecycle behavior.

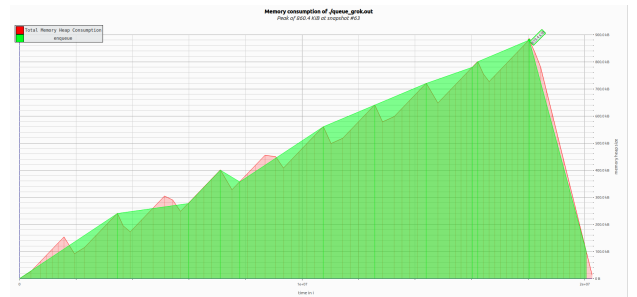


Fig. 5: Massif output for *queue\_grok*

The Massif graph visibly oscillates because allocations and deallocations are interleaved dynamically.

Latency results:

| Workload | Avg Latency (ns) |
|----------|------------------|
| 100K     | 45               |
| 1M       | 31               |
| 10M      | 29               |

queue\_grok latency

queue\_grok demonstrates:

- Efficient allocator reuse
- Heap recycling
- Allocator warm-up
- Improved scaling behavior

#### E. Comparison: Queue Implementations

| Metric       | GPT    | Claude | Grok      |
|--------------|--------|--------|-----------|
| 100K Latency | 46 ns  | 85 ns  | 45 ns     |
| 1M Latency   | 51 ns  | 42 ns  | 31 ns     |
| 10M Latency  | ~30 ns | 41 ns  | 29 ns     |
| Trend        | Stable | Stable | Improving |

Queue comparison

All queue implementations logically followed the same enqueue/dequeue workload pattern, but implementation-level differences produced different runtime characteristics.

This demonstrates that identical algorithms can still produce very different allocator and hardware behavior depending on instrumentation and implementation choices.

#### F. Workload Analysis

| Vertices | Edges   | Avg Latency (ns) | Result     |
|----------|---------|------------------|------------|
| 500      | 50000   | 68.71            | Successful |
| 5000     | 500000  | 41.20            | Successful |
| 50000    | 5000000 | 34.56            | Successful |

Adjacency list scaling

1) *Adjacency List Scaling*: The adjacency list scales efficiently because memory growth is approximately linear with the number of edges.

Larger workloads produced more stable latency measurements because allocator reuse improved and statistical noise was reduced.

| Vertices | Edges   | Avg Latency (ns) | Result             |
|----------|---------|------------------|--------------------|
| 500      | 50000   | 1505             | Successful         |
| 5000     | 500000  | 19428            | Successful         |
| 50000    | 5000000 | —                | Segmentation Fault |

Adjacency matrix scaling

2) *Adjacency Matrix Scaling*: Memory consumption increases quadratically with vertices.

Large contiguous allocations eventually become infeasible because the operating system cannot efficiently satisfy multi-gigabyte requests.

| Workload | GPT (ns) | Claude (ns) | Grok (ns) |
|----------|----------|-------------|-----------|
| 100K     | 46       | 85          | 45        |
| 1M       | 51       | 42          | 31        |
| 10M      | ~30      | 41          | 29        |

Queue workload scaling

3) *Queue Scaling*: As workloads increase:

- Allocator freelists become populated
- Memory reuse improves
- Latency stabilizes
- Statistical variability decreases

#### G. Variability and Stability

Latency measurements in modern systems are probabilistic rather than deterministic.

The latency distributions observed during experimentation exhibited heavy-tailed behavior. Most allocations completed extremely quickly, while a small number of allocations incurred very large delays.

This suggests that allocator latency is bimodal rather than normally distributed. Fast-path allocations are typically served directly from allocator freelists or tcaches, while slower allocations may trigger heap expansion, page faults, allocator synchronization or virtual-memory operations.

The largest observed latency spikes exceeded 400000 ns, which likely indicates slow-path allocator behavior involving page allocation or kernel-level memory-management operations rather than ordinary allocation overhead.

| Run | Avg Latency (ns) | Max Latency (ns) |
|-----|------------------|------------------|
| 1   | 41.20            | 211558           |
| 2   | 68.71            | 49173            |
| 3   | 34.56            | 117141           |
| 4   | 45.80            | 102561           |
| 5   | 76.95            | 224830           |
| 6   | 78.30            | 392143           |
| 7   | 80.67            | 224830           |
| 8   | 78.36            | 436165           |
| 9   | 78.77            | 391952           |
| 10  | 77.20            | 202057           |

Adjacency list variability

1) *Adjacency List Variability*: Average latency remains relatively stable despite extremely large spikes.

These spikes likely originate from:

- Page faults
- Heap expansion
- Kernel scheduling
- Allocator synchronization

2) *Adjacency Matrix Variability*: Variability is smaller because fewer allocations reduce statistical noise.

3) *Effect of Sample Size*: Large workloads reduce statistical noise because averages are computed over many more samples.

This demonstrates that single-allocation measurements are unreliable for allocator-performance analysis.

| Run | Avg Latency (ns) |
|-----|------------------|
| 1   | 22767            |
| 2   | 19730            |
| 3   | 21478            |
| 4   | 22525            |
| 5   | 24516            |
| 6   | 21402            |
| 7   | 19296            |
| 8   | 19089            |
| 9   | 24452            |
| 10  | 19017            |

Adjacency matrix variability

| Case                    | Variability    |
|-------------------------|----------------|
| adj_mat (1 malloc)      | Very High      |
| adj_mat (10 mallocs)    | Reduced        |
| adj_list (500K mallocs) | Stable Average |

Effect of allocation count on variability

## H. Hardware-Level Analysis

Hardware-level behavior was analyzed using perf.

We hypothesized that the adjacency matrix would achieve lower cache-miss rates because contiguous memory traversal improves spatial locality and hardware prefetching efficiency. The perf results confirmed this expectation, with the adjacency matrix producing the lowest cache-miss percentage among all workloads.

However, it also produced the largest instruction count because matrix traversal requires processing  $O(V^2)$  elements.

This demonstrates an important systems insight: cache efficiency alone does not determine performance.

Total computational workload also matters.

The adjacency list suffered slightly higher cache misses due to pointer chasing, but significantly lower instruction counts allowed much faster execution.

Although the adjacency matrix achieved superior cache locality, its computational workload remained extremely large because matrix traversal requires processing every possible vertex pair.

This demonstrates that cache efficiency alone does not determine runtime performance. Algorithmic complexity and total instruction count are equally important.

The adjacency list therefore represents an interesting trade-off: slightly worse locality but dramatically reduced computational workload for sparse graphs.

queue\_claude produced very high instruction counts because instrumentation overhead dominated execution.

This demonstrates that measurement infrastructure itself can substantially affect runtime behavior.

## I. Final Comparison

The adjacency list is memory efficient and scalable but suffers from fragmentation and locality problems.

The adjacency matrix provides excellent locality but scales poorly due to massive memory requirements.

The queue workloads reveal allocator reuse behavior and show how allocator performance evolves dynamically under repeated allocations and deallocations.

| Program          | Instructions | Cache Miss (%) | Time (s) |
|------------------|--------------|----------------|----------|
| Adjacency Matrix | 8.7B         | 5.07           | 0.86     |
| Adjacency List   | 343M         | 6.37           | 0.046    |
| Queue GPT        | 56M          | 12.2           | 0.011    |
| Queue Claude     | 740M         | 5.96           | 0.25     |
| Queue Grok       | 54M          | 10.4           | 0.010    |

Hardware-level performance comparison using perf counters

| Metric          | Adj List   | Matrix    | Queue     |
|-----------------|------------|-----------|-----------|
| Allocation Type | Many Small | Few Large | Dynamic   |
| Memory Growth   | Gradual    | Sudden    | Mixed     |
| Peak Memory     | Low        | High      | Moderate  |
| Overhead        | High       | Low       | Moderate  |
| Locality        | Poor       | Good      | Moderate  |
| Scalability     | Good       | Poor      | Very Good |

Overall comparison and allocation-strategy tradeoffs

No single allocation strategy is universally optimal. The best choice depends on workload characteristics, scalability requirements and memory-access behavior.

## J. Practical Allocation Tradeoffs

The experiments suggest that different allocation strategies are preferable under different workload conditions.

Adjacency lists are more suitable for sparse graphs and large-scale workloads because memory usage grows approximately linearly with the number of edges. This makes them significantly more scalable for real-world sparse graph applications.

Adjacency matrices are better suited for dense graphs or workloads requiring frequent random access because contiguous memory improves locality, traversal predictability and hardware prefetching efficiency.

Dynamic queue workloads benefit strongly from allocator warmup and freelist reuse. Systems performing repeated small-object allocations may therefore benefit from memory pools, slab-style allocators or custom object caches that minimize allocator overhead.

## VI. CONCLUSION

This project analyzed memory allocation behavior using both Valgrind and perf.

Instead of focusing only on final memory usage, the project studied how memory evolves dynamically during execution.

The adjacency list demonstrated fragmented pointer-heavy allocation behavior involving hundreds of thousands of small allocations. It achieved good scalability and memory efficiency but suffered from fragmentation, allocator overhead and reduced locality.

The adjacency matrix demonstrated contiguous allocation behavior with excellent locality and stable heap behavior. However, its memory requirements grew quadratically, making it unsuitable for very large graphs.

The queue programs simulated dynamic allocation lifecycles and revealed allocator warm-up, memory reuse and heap recycling behavior. queue\_grok showed the best large-workload scaling, while queue\_claude demonstrated how instrumentation overhead can significantly alter runtime behavior.



Hardware-level analysis revealed that low cache-miss rates alone do not guarantee better performance. Instruction count and total computational workload are equally important.

The combined use of Valgrind and perf provided a much deeper systems-level understanding than either tool could provide independently.

Overall, the experiments demonstrate that memory allocation is not merely a storage mechanism. Allocation strategy fundamentally shapes fragmentation, locality, scalability, variability and hardware-level execution behavior. Different allocation strategies therefore change how software interacts with modern operating systems, allocators and hardware architectures.

#### REFERENCES

- [1] N. Nethercote and J. Seward, “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation,” 2007.
- [2] N. Nethercote and J. Seward, “How to Shadow Every Byte of Memory Used by a Program,” 2007.
- [3] Valgrind Documentation: <https://valgrind.org>
- [4] Linux perf Documentation: <https://perf.wiki.kernel.org>